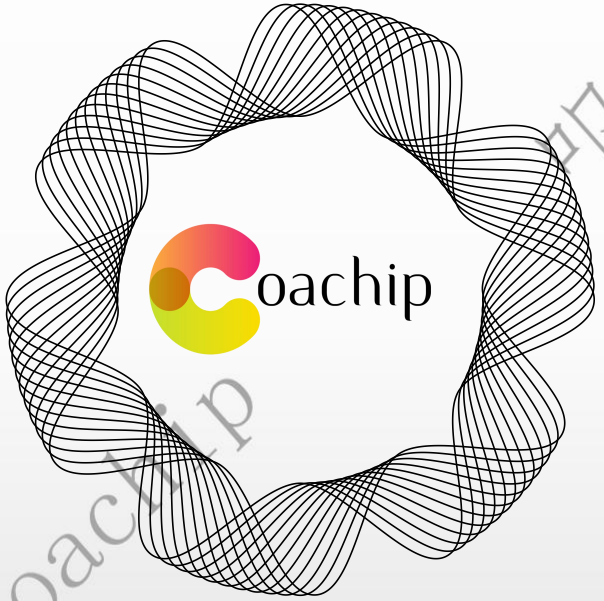


SV面向对象编程

叩特电子Coachip

课程内容及目标

课程内容：

- 包的意义
- 包的定义
- 包与库的区别
- 包的命名规则

目标：

- 了解包的定义及使用



叩特电子Coachip

包的意义

- SV语言提供了一种在多个module、interface和program之中共享parameter、data、type、task、function、class等的方法，即利用package（包）的方式来实现。如果用装修一个大房子（完整的验证环境）来看的话，我们喜欢将不同模块的类定义归整到不同的package中。
- 这么做的好处在于将一簇相关的类组织在了单一的命名空间（namespace）下，使得分属于不同模块验证环境的类来自于不同的package，这样便可以通过package来解决类的归属问题。

叩特电子Coachip

包的定义

- package regs_pkg和arb_pkg的定义

```
package regs_pkg;
  `include "stimulator.sv"
  `include "monitor.sv"
  `include "checker.sv"
  `include "env.sv"
endpackage

package arb_pkg;
  `include "stimulator.sv"
  `include "monitor.sv"
  `include "checker.sv"
  `include "env.sv"
endpackage
```

叩特电子Coachip

包的定义

- 两位verifier在各自的package：regs_pkg和arb_pkg中都定义了4个与模块验证相关的类即stimulator、monitor、checker和env。但是两个package中同名的类，它们的内容是不相同的，实现的也是不同的功能。
- 如果将这些重名的类归属到不同的package中编译，有没有问题呢？会不会发生重名的编译冲突？不要担心，**package会将命名空间分隔开来**，这样如果要使用不同package中的同名类，他们只需要**注明要使用哪一个package**中的类具体操作如下所示：

```
module arbiter_tb;
  regs_pkg::monitor mon1 = new();
  arb_pkg::monitor mon2 = new();
endmodule
```

即特电子Coachip

包的命名规则

- 创建package的时候，已经在指定包名称的时候隐含地指定了包的默认路径，即包文件所在的路径。如果有其它要被包含在包内的文件在默认路径之外，需要在编译包的时候加上**额外指定的搜寻路径选项“+incdir+PATH”**。
- 实际工作中的编码风格要求定义的**package名称独一无二**，其内部定义的类也应该尽可能地独一无二。例如，上例regs_pkg和arb_pkg中有同名的类，类的名称加上说明前缀，对代码的可读性及后面的处理会更好。
- 不同package中定义的类名也不相同时，在顶层的引用也可以通过**“import pkg_name::*”**的形式，来表示在module arbiter_tb中引用的类如果在当前域（arbiter内部）中没有定义的话，会搜寻regs_pkg和arb_pkg中定义的类，**又由于它们各自包含的类名不相同，因此也无需担心下面的搜寻会遇到同名类发生冲突的问题**。

即特电子 Coachip

包的命名规则

```
package regs_pkg;
  `include "regs_stm.sv"
  `include "regs_mon.sv"
  `include "regs_chk.sv"
  `include "regs_env.sv“
endpackage

package arb_pkg;
  `include "arb_stm.sv"
  `include "arb_mon.sv"
  `include "arb_chk.sv"
  `include "arb_env.sv"
endpackage
```

```
module test_tb;
  import regs_pkg::*;
  import arb_pkg::*;
  regs_mon mon1= new();
  arb_mon mon2 = new();
endmodule
```

叩特电子Coachip

System Verilog包的使用

- 包中可以定义类、静态方法和静态变量
- 如果将类封装在某一个包中，那么它就不应该在其它地方编译，这么做的好处在于之后对类的引用更加方便。
- 类和包是好朋友，包是类的归宿，类是包的子民。
- 一个完整模块的验证环境组件类，应该由一个对应的模块包来封装。
- 使用`include的关键词完成类在包中的封装，要注意编译的前后顺序来放置各个`include的类文件。
- 编译一个包的背后实际是将各个类文件“平铺”在包中，按照顺序完成包和各个类的有序编译。
- 实用的类可以通过‘import’完成包中所有类或者某一个类的导入，使得新的环境可以识别出该类，否则类会躺在包这个盒子里不被外部识别。

叩特电子Coachip

System Verilog包的使用

- 通常情况下，一般在文件开头的位置加上：

import pkg_name::*　//package中的所有元素都可以被当前文件直接调用。

```
import regs_pkg::regs_mon mon1= new();
import arb_pkg::arb_mon mon2 = new();

regs_mon mon1= new();
arb_mon mon2 = new();
```

叩特电子Coachip

Q&A

昂特电子Coachip